# Bambu Printer Status
# JSON Setup Instructions

Updated October 16, 2026 | Tidbyt / Tronbyt | Author: Robert Ison

Set up a Raspberry Pi to generate a status.json feed, make it available to Tronbyt, and configure the updated Bambu Printer Status applet. This edition documents the new app features while retaining the Python updater from the supplied original guide.

## Already have a working status feed?

Do not reinstall Python, replace your updater, add a second cron job, or start a second web server. Keep your existing status URL and printer selection. Install the updated applet and review Section 7. No JSON schema change is required for the new finish-time and recent-completion features.

## What changed in the app

• Estimated local finish time alongside remaining duration, with Tomorrow or a date when needed.

• A Finished message for 30 minutes after the recorded completion time, when enabled.

• Optional old-status-file warnings based on updated_at: warning at 5 minutes; STALE DATA at 15 minutes.

• A Show Setup Instructions toggle with three staggered scrolling instruction rows and a reference to this PDF.

• Automatic 1x/2x layout, retaining the four-row design and staggered job/footer scrolling.

• Safer selected-printer handling, duplicate-name cleanup, and guards against negative durations.

## Important scope and limitations

The Python updater in Appendix A is reproduced from the original PDF, not redesigned. Its syntax was checked during document preparation; its cloud API endpoints and authentication were not tested live. The full updated .star applet is supplied separately in the conversation/app distribution; this PDF is not a substitute for that file.

New users need a valid Bambu cloud access token before completing setup. This guide does not provide a verified token-acquisition or renewal procedure. Account sign-in methods and region-specific API behavior may differ. Do not assume a password, serial number, or LAN access code is the required cloud token.

# 1. Prepare the Raspberry Pi

Run the Linux commands in a terminal on the Pi, or in an SSH session connected to it. Replace your_username with the actual Linux username. Do not paste the shell prompt itself.

```
ssh your_username@raspberrypi.local
```

Use your Pi IP address instead of raspberrypi.local if hostname resolution does not work. For a fresh Raspberry Pi OS setup, install Python tools and use a virtual environment:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl
mkdir -p ~/bambu-cloud
python3 -m venv ~/bambu-cloud/.venv
~/bambu-cloud/.venv/bin/python -m pip install requests
```

The updater, status history, and generated JSON live in ~/bambu-cloud. Keep the cloud token in your home directory, outside any directory you serve over HTTP.

# 2. Create the token file

```
nano ~/.bambu_token
```

Paste only the valid access-token string. Save with Ctrl+O, press Enter, and exit with Ctrl+X. The original updater also accepts JSON containing a token or access_token property.

```
chmod 600 ~/.bambu_token
```

Never publish this file, include it in screenshots, or put it in a public code repository. Do not serve your entire home directory over HTTP.

# 3. Install and test the updater

For a new installation, create the file below and paste the complete Python source from Appendix A. Do not include the appendix heading or page numbers.

```
nano ~/bambu-cloud/cloud_status_update.py
```

Then check syntax and run it using the virtual environment:

```
~/bambu-cloud/.venv/bin/python -m py_compile \
  ~/bambu-cloud/cloud_status_update.py
~/bambu-cloud/.venv/bin/python \
  ~/bambu-cloud/cloud_status_update.py
python3 -m json.tool ~/bambu-cloud/status.json
```

Confirm that printers contains the expected printer IDs and names. Review probe_errors at both the top level and inside printer entries. A Wrote status.json message alone is not proof that the cloud calls succeeded.

# 4. Schedule updates once per minute

First inspect existing jobs. If an updater job already exists and works, keep it rather than adding a duplicate.

```
crontab -l
crontab -e
```

For a fresh setup, add the following as one physical line, replacing your_username in both paths:

```
* * * * * /home/your_username/bambu-cloud/.venv/bin/python /home/your_username/bambu-cloud/cloud_status_update.py >/tmp/bambu-cloud.log
```

The line above is long: copy it as a single line. Wait a minute, then inspect updated_at and /tmp/bambu-cloud.log. Running the updater repeatedly at the same time can race on the shared JSON file; avoid duplicate schedules or simultaneous manual runs.

# 5. Make status.json available over HTTP

Skip this section if your existing URL already works. The example below serves a dedicated public directory containing a copy of status.json, not your token, virtual environment, or updater source.

```
mkdir -p ~/bambu-public
cp ~/bambu-cloud/status.json ~/bambu-public/status.json
```

For a NEW cron setup using this dedicated directory, replace the updater-only cron line from Section 4 with the following single line. The publication copy happens after the updater exits successfully. It does not verify cloud API success.

```
* * * * * /home/your_username/bambu-cloud/.venv/bin/python /home/your_username/bambu-cloud/cloud_status_update.py >/tmp/bambu-cloud.log
```

Try the server temporarily in a terminal:

```
python3 -m http.server 8787 --bind 0.0.0.0 \
  --directory "$HOME/bambu-public"
```

Leave that terminal open for the test. From another device on your trusted home network, open http://PI_IP:8787/status.json, replacing PI_IP with the Pi address. From a separate Pi terminal, test:

```
curl -f http://127.0.0.1:8787/status.json
```

Python http.server provides basic HTTP file serving, without authentication or encryption. Use it only on a trusted local network for this example. Do not expose port 8787 to the internet. A public/cloud-hosted Tronbyt server cannot automatically reach a private home-network URL.

# 6. Optional: start the HTTP server after reboot

Use only if no existing service manages port 8787. Stop the temporary server with Ctrl+C first. Create a systemd service:

```
sudo nano /etc/systemd/system/bambu-status-http.service
```

Paste the following, replacing your_username in both places:

```
[Unit]
Description=Bambu status JSON HTTP server
After=network.target

[Service]
Type=simple
User=your_username
ExecStart=/usr/bin/python3 -m http.server 8787 --bind 0.0.0.0 --directory /home/your_username/bambu-public
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now bambu-status-http.service
sudo systemctl status bambu-status-http.service
```

A bind/address-in-use error usually means another server is already using that port. Do not keep launching additional servers; identify and reuse the existing one.

# 7. Configure the updated applet

Use the full updated bambuprinterstatus.star script supplied with the app. Its final instructions-screen function uses explicit + operators between string fragments; Python-style adjacent quoted strings are not accepted by the Starlark parser used in this setup.

## Show Setup Instructions

Off for normal status. On replaces status with a heading and three staggered scrolling setup rows. This takes priority over active-only filtering and works without a status URL during rendering.

## Status JSON URL

Enter the complete reachable URL, for example http://PI_IP:8787/status.json. Use the same URL you verified in a browser.

## Printer

Choose the desired printer from the generated dropdown. If that ID later disappears, the app will not silently display another printer.

## Scroll

Slow, Medium, or Fast. The default is Slow.

## Only During Active Prints

Shows PRINTING and PAUSED states only. Idle, offline, and recently finished states are skipped. Configuration and HTTP error screens can still appear.

## Estimated Finish Time

Adds an estimated local clock time when time_left is present and positive. At zero remaining time, shows Finishing / awaiting update.

## Highlight Recent Completion

When IDLE, shows Finished for 30 minutes after last_completed_end_time. Active-only filtering takes precedence.

## Warn About Old Data

Uses printer-level updated_at if present, otherwise the top-level updated_at. No timestamp means no stale warning.

# 8. Layout and PDF reference

Normal mode keeps printer name and headline static. Job details scroll on row 3. The footer scrolls on row 4 with offset_start based on the length of row 3. Setup mode uses one static heading and three instruction marquees with cumulative offsets.

The script automatically selects tb-8 at 64 pixels wide, or terminus-16 at 128 pixels wide when canvas.is2x() is true. Approximate font-width values of 5 and 8 control the stagger. This preserves the design, but does not guarantee exact timing or pixel-perfect text measurement.

The finish-time display uses config.get("$tz") when supplied, falling back to America/New_York. Check your device/server timezone if the displayed clock time is wrong.

The full guide title appears in configuration descriptions. SETUP_GUIDE_URL is intentionally empty until the publisher supplies a public PDF URL. An attachment in a private conversation is not a public distribution link. On-screen URLs are text, not clickable controls.

# 9. What the estimates and warnings mean

• Progress and remaining duration: the original Python code estimates them from task startTime and costTime. They are not direct layer-by-layer progress telemetry, and pauses or cloud delays may reduce accuracy.

• Finish time: calculated at render time from now plus time_left. Old remaining-time values can produce a drifting finish estimate until refreshed.

• Recent completion: based on recorded history, not independently verified successful completion. The retained updater can update history when PRINTING changes to PAUSED or OFFLINE as well as IDLE.

• Old file warning: at 5 minutes the footer warns; at 15 minutes the headline becomes STALE DATA. The original updater refreshes updated_at even on handled API failures, so this does not certify fresh printer readings.

• Unsupported extra telemetry: this edition does not add temperatures, layer counts, filament status, or detailed pause/error reasons to your JSON producer.

# 10. Troubleshooting

## No status URL

Enter the complete URL or enable Show Setup Instructions.

## HTTP error / cannot connect

Verify the Pi address, port 8787, web-server service, and local network reachability. A Docker-hosted Tronbyt instance must also be able to reach the Pi URL.

## No printers / selected printer not found

Inspect status.json and probe_errors. Confirm the token and cloud request results; then reselect the printer if its configured ID is no longer present.

## File is fresh but printer status is wrong

updated_at can advance despite handled API errors. Read probe_errors and the updater log. Do not treat freshness alone as successful telemetry retrieval.

## Wrong finish time

Check device timezone and the accuracy/age of time_left. The upstream estimate may not account for pauses or other delays.

## Applet fails to render

Inspect the exact Starlark error and line number. Malformed JSON or timestamp strings can still fail rendering. Keep a known-working .star backup.

## Instructions stop before the end

This can indicate animation duration or output-size limits, not a fixed character limit. The updated instructions use shorter rows; preserve the intended stagger when tuning playback.

```
cat /tmp/bambu-cloud.log
python3 -m json.tool ~/bambu-cloud/status.json
sudo journalctl -u bambu-status-http.service -n 40 --no-pager
```

# 11. Distribution checklist

• Ship the updated .star applet alongside this PDF.

• For new users, supply a maintained and verified way to obtain/renew the required cloud token. Token acquisition remains outside this guide.

• If using a hosted PDF link, set SETUP_GUIDE_URL in the applet before distributing it.

• Do not bundle your own token, printer identifiers, personal status file, or private credentials.

• Preserve the known limitations of the unchanged updater; do not describe its estimates as guaranteed real-time printer measurements.

# Documentation basis

Updater source: the original user-supplied Bambu Printer Status JSON Setup Instructions PDF. The new app behavior and settings are based on the updated Starlark script developed in this conversation.

HTTP serving and security: https://docs.python.org/3/library/http.server.html

Service restart behavior: https://www.freedesktop.org/software/systemd/man/systemd.service.html

Validation: Appendix A was extracted from the original PDF and passed Python compile() syntax validation. No cloud requests, account authentication, printer connection, or end-to-end app render were executed while preparing this edition.

# Appendix A. Original Python updater

This is the original updater source, preserved without functional changes. Save only the code as ~/bambu-cloud/cloud_status_update.py. It retains the limitations described in Section 9. Installation instructions above use a virtual environment rather than changing the script.

```
#!/usr/bin/env python3
import json
from datetime import datetime, timezone
from pathlib import Path
import requests
STATUS_PATH = Path.home() / "bambu-cloud" / "status.json"
TOKEN_PATH = Path.home() / ".bambu_token"
API_BASE = "https://api.bambulab.com"
def iso_now():
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat()
def read_token():
    raw = TOKEN_PATH.read_text().strip()
    if raw.startswith("{"):
        try:
            data = json.loads(raw)
            t = data.get("token") or data.get("access_token")
            if t:
                return t.strip()
        except Exception:
            pass
    return raw
def api_get(path, token, params=None):
    url = f"{API_BASE}{path}"
    headers = {
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
    }
    resp = requests.get(url, headers=headers, params=params, timeout=15)
    resp.raise_for_status()
    return resp.json()
def fmt_duration(seconds):
    if seconds is None:
        return None
    try:
        seconds = int(seconds)
    except Exception:
        return None
    if seconds < 60:
        return f"{seconds}s"
    minutes, _ = divmod(seconds, 60)
    if minutes < 60:
        return f"{minutes}m"
    hours, minutes = divmod(minutes, 60)
    if minutes == 0:
        return f"{hours}h"
    return f"{hours}h {minutes}m"
def normalize_state(online, raw_status, cloud_print_job):
    if not online:
        return "OFFLINE"
    status = (raw_status or "").upper()
    if status in ("RUNNING", "PRINTING"):
        return "PRINTING"
    if status in ("PAUSE", "PAUSED"):
        return "PAUSED"
    if status in ("SUCCESS", "FINISH", "COMPLETED", "IDLE", ""):
        return "IDLE"
    if cloud_print_job:
        return "IDLE"
    return status
def list_printers(token):

    data = api_get("/v1/iot-service/api/user/bind", token)
    return data.get("devices") or data.get("data") or []
def fetch_tasks(token):
```

# Appendix A. Python updater (continued)

```
    data = api_get("/v1/user-service/my/tasks", token)
    hits = data.get("hits")
    return hits if isinstance(hits, list) else []
def index_tasks_by_device_and_id(tasks):
    tasks_by_id = {}
    tasks_by_device = {}
    for t in tasks:
        if not isinstance(t, dict):
            continue
        device_id = t.get("deviceId")
        job_id = t.get("id")
        if not device_id:
            continue
        if job_id is not None:
            try:
                tasks_by_id[(str(device_id), int(job_id))] = t
            except Exception:
                pass
        tasks_by_device.setdefault(str(device_id), []).append(t)
    for dev_id, lst in tasks_by_device.items():
        lst.sort(key=lambda task: task.get("startTime") or "", reverse=True)
    return tasks_by_id, tasks_by_device
def pick_task_for_printer(device_id, cloud_print_job, tasks_by_id, tasks_by_device):
    dev_key = str(device_id)
    if cloud_print_job is not None:
        try:
            hit = tasks_by_id.get((dev_key, int(cloud_print_job)))
            if hit:
                return hit
        except Exception:
            pass
    dev_tasks = tasks_by_device.get(dev_key) or []
    if dev_tasks:
        return dev_tasks[0]
    return None
def extract_job_fields_from_task(task):
    if not isinstance(task, dict):
        return None, None, None, None, None
    job_name = task.get("title") or task.get("designTitle") or None
    job_objects = task.get("designTitle") or None
    estimated_seconds = task.get("costTime")
    estimated_text = fmt_duration(estimated_seconds)
    end_time = task.get("endTime") or None
    return job_name, job_objects, estimated_seconds, estimated_text, end_time
def load_previous_status():
    if not STATUS_PATH.exists():
        return {}
    try:
        data = json.loads(STATUS_PATH.read_text())
        printers = data.get("printers", [])
        out = {}
        for p in printers:
            pid = p.get("id")
            if pid:
                out[str(pid)] = p

        return out
    except Exception:
        return {}
def build_printer_record(device, task, previous, now_iso):
    printer_id = str(device.get("dev_id") or device.get("id") or device.get("sn") or "")
    printer_name = device.get("name") or device.get("dev_name") or device.get("device_name") or printer_id
    online = bool(device.get("online", False))
    raw_status = device.get("print_status") or device.get("status") or device.get("stg_cur") or ""
```

# Appendix A. Python updater (continued)

```
    cloud_print_job = device.get("print_job")
    state = normalize_state(online, raw_status, cloud_print_job)
    job_name, job_objects, est_seconds, est_text, task_end_time = extract_job_fields_from_task(task)
    progress = None
    time_left = None
    if state == "PRINTING" and task is not None:
        start_str = task.get("startTime")
        if start_str:
            try:
                ts = start_str.replace("Z", "+00:00")
                start_dt = datetime.fromisoformat(ts)
                now_dt = datetime.now(timezone.utc)
                elapsed = max((now_dt - start_dt).total_seconds(), 0)
                if est_seconds:
                    total = max(int(est_seconds), 1)
                    progress = min(100, max(0, int(elapsed * 100 // total)))
                    time_left = max(total - int(elapsed), 0)
            except Exception:
                pass
    prev_state = (previous or {}).get("state")
    prev_job = (previous or {}).get("cloud_print_job")
    prev_idle_since = (previous or {}).get("idle_since")
    prev_last_completed_job_id = (previous or {}).get("last_completed_job_id")
    prev_last_completed_job_name = (previous or {}).get("last_completed_job_name")
    prev_last_completed_job_objects = (previous or {}).get("last_completed_job_objects")
    prev_last_completed_end_time = (previous or {}).get("last_completed_end_time")
    prev_last_completed_duration_seconds = (previous or {}).get("last_completed_duration_seconds")
    prev_last_completed_duration_text = (previous or {}).get("last_completed_duration_text")
    idle_since = None
    last_completed_job_id = prev_last_completed_job_id
    last_completed_job_name = prev_last_completed_job_name
    last_completed_job_objects = prev_last_completed_job_objects
    last_completed_end_time = prev_last_completed_end_time
    last_completed_duration_seconds = prev_last_completed_duration_seconds
    last_completed_duration_text = prev_last_completed_duration_text
    if state == "PRINTING":
        idle_since = None
    else:
        if prev_state == "PRINTING":
            idle_since = now_iso
            if cloud_print_job is not None:
                last_completed_job_id = cloud_print_job
            elif prev_job is not None:
                last_completed_job_id = prev_job
            if job_name:
                last_completed_job_name = job_name
            if job_objects:
                last_completed_job_objects = job_objects
            if task_end_time:
                last_completed_end_time = task_end_time
            elif not last_completed_end_time:
                last_completed_end_time = now_iso
            if est_seconds is not None:
                last_completed_duration_seconds = est_seconds
                last_completed_duration_text = est_text
        elif prev_state in ("IDLE", "PAUSED", "OFFLINE") and prev_idle_since not in (None, ""):

            idle_since = prev_idle_since
        else:
            idle_since = None
    if last_completed_job_id is None and cloud_print_job is not None:
        last_completed_job_id = cloud_print_job
    if last_completed_job_name is None and job_name is not None:
        last_completed_job_name = job_name
```

# Appendix A. Python updater (continued)

```
    if last_completed_job_objects is None and job_objects is not None:
        last_completed_job_objects = job_objects
    if last_completed_end_time is None and task_end_time is not None:
        last_completed_end_time = task_end_time
    if last_completed_duration_seconds is None and est_seconds is not None:
        last_completed_duration_seconds = est_seconds
        last_completed_duration_text = est_text
    if state == "PRINTING":
        display_job_name = job_name
        display_job_objects = job_objects
        display_est_seconds = est_seconds
        display_est_text = est_text
    else:
        display_job_name = last_completed_job_name or job_name
        display_job_objects = last_completed_job_objects or job_objects
        display_est_seconds = last_completed_duration_seconds if last_completed_duration_seconds is not None else est_seconds
        display_est_text = last_completed_duration_text or est_text
    return {
        "id": printer_id,
        "name": printer_name,
        "online": online,
        "state": state,
        "label": raw_status,
        "cloud_print_status": raw_status,
        "cloud_print_job": cloud_print_job,
        "job_name": display_job_name,
        "job_objects": display_job_objects,
        "progress": progress,
        "time_left": time_left,
        "estimated_duration_seconds": display_est_seconds,
        "estimated_duration_text": display_est_text,
        "idle_since": idle_since,
        "last_completed_job_id": last_completed_job_id,
        "last_completed_job_name": last_completed_job_name,
        "last_completed_job_objects": last_completed_job_objects,
        "last_completed_end_time": last_completed_end_time,
        "last_completed_duration_seconds": last_completed_duration_seconds,
        "last_completed_duration_text": last_completed_duration_text,
        "probe_errors": [],
    }
def main():
    token = read_token()
    printers_out = []
    top_errors = []
    now_iso = iso_now()
    previous_by_id = load_previous_status()
    try:
        devices = list_printers(token)
    except Exception as e:
        devices = []
        top_errors.append(f"list_printers: {repr(e)}")
    try:
        tasks = fetch_tasks(token)
        tasks_by_id, tasks_by_device = index_tasks_by_device_and_id(tasks)
    except Exception as e:
        tasks_by_id, tasks_by_device = {}, {}
        top_errors.append(f"fetch_tasks: {repr(e)}")
    for device in devices:
        try:
            printer_id = str(device.get("dev_id") or device.get("id") or device.get("sn") or "")
            cloud_print_job = device.get("print_job")

            task = pick_task_for_printer(printer_id, cloud_print_job, tasks_by_id, tasks_by_device)
            previous = previous_by_id.get(printer_id, {})
```

# Appendix A. Python updater (continued)

```
            printers_out.append(build_printer_record(device, task, previous, now_iso))
        except Exception as e:
            printers_out.append({
                "id": str(device.get("dev_id") or "unknown"),
                "name": device.get("name") or "Unknown Printer",
                "online": False,
                "state": "ERROR",
                "label": "ERROR",
                "cloud_print_status": None,
                "cloud_print_job": None,
                "job_name": None,
                "job_objects": None,
                "progress": None,
                "time_left": None,
                "estimated_duration_seconds": None,
                "estimated_duration_text": None,
                "idle_since": None,
                "last_completed_job_id": None,
                "last_completed_job_name": None,
                "last_completed_job_objects": None,
                "last_completed_end_time": None,
                "last_completed_duration_seconds": None,
                "last_completed_duration_text": None,
                "probe_errors": [f"build_printer_record: {repr(e)}"],
            })
    payload = {
        "updated_at": now_iso,
        "printers": printers_out,
        "probe_errors": top_errors,
    }
    STATUS_PATH.write_text(json.dumps(payload, indent=2))
    print(f"Wrote {STATUS_PATH} with {len(printers_out)} printers")
if __name__ == "__main__":
    main()
```